Coding Problem 1 : The Galactic Cargo Manifest Processor

Scenario Background:

You are the lead software engineer for the *Andromeda Freight Corporation*. Your company manages a massive fleet of interstellar cargo ships. Each ship carries thousands of individual cargo containers.

You need to build a data processing pipeline that analyzes the cargo manifests. The system needs to filter out hazardous materials that are improperly shielded, calculate the total value of safe cargo, and map the remaining cargo into a highly compressed transmission format to be beamed back to Earth headquarters.

Because the data is vast, the system must be built using **Java 8+ Streams and Functional Interfaces** for maximum efficiency and parallelization potential.

Implementation Constraints & Requirements:

1. Inheritance and Cargo Structure:

- Create an abstract base class Cargo. It must contain:
    - String containerId
    - double valueInCredits
    - boolean isHazardous
- Create two concrete classes that extend Cargo:
    - StandardCargo: (Nothing special, inherits fields).
    - BiologicalCargo: Contains an additional field boolean isShielded.

2. Custom Functional Interfaces:

You cannot solely rely on built-in Java functional interfaces (Predicate, Function, etc.) for everything. You must define **two custom functional interfaces**:

- @FunctionalInterface CargoInspector: Must contain a method that takes a Cargo object and returns a boolean (indicating if the cargo is safe for transport).
- @FunctionalInterface CargoCompressor: Must contain a method that takes a Cargo object and returns a String (representing the compressed telemetry data).

3. The Processing Engine (Stream Pipeline):

Create a class ManifestProcessor with a method:

public List<String> processManifest(List<Cargo> manifest, CargoInspector inspector, CargoCompressor compressor)

Inside this method, you must write a single, chained Java Stream pipeline that performs the following steps in exact order:

1. **Filter:** Use the provided CargoInspector to filter out any cargo that is deemed unsafe.
2. **Filter:** Filter out any cargo that has a valueInCredits less than 1000.0 (deemed not worth the fuel to transport).
3. **Map:** Use the provided CargoCompressor to transform the remaining Cargo objects into their String telemetry format.
4. **Collect:** Return the final list of strings.

4. The Business Logic Rules (Main Execution):

In your main method, you must instantiate the custom functional interfaces using **Lambda Expressions** based on these strict business rules:

- **The Inspector Lambda:** A Cargo is considered *unsafe* (returns false) if it is isHazardous == true AND it is an instance of BiologicalCargo where isShielded == false. All other cargo is safe (returns true).
- **The Compressor Lambda:** The compressed string must be formatted exactly as: "[ID]-[V]" where [ID] is the first 4 characters of the containerId and [V] is the integer value of the credits. (e.g., Container "ALPHA-99" with value 5000.50 becomes "ALPH-5000").

Why this makes a great interview/study question:

1. **Validating @FunctionalInterface:** It forces the developer to understand that functional interfaces are just standard interfaces with exactly one abstract method. By creating custom ones, they prove they understand the mechanics underneath standard JDK interfaces like Predicate or Function.
2. **Lambda Syntax & Polymorphism:** The safetyInspector lambda requires the developer to use instanceof and downcasting within a functional block. This tests their ability to mix traditional OOP polymorphism with modern functional paradigms.
3. **Stream Anatomy:** The processManifest method tests the strict anatomical requirements of a Stream pipeline: creating the stream, applying intermediate operations (filter, map), and executing a terminal operation (collect). It also demonstrates how custom lambdas can be seamlessly injected into standard Stream methods.

Coding Problem 2: The "NeuroLink" Memory Engram Sorter Scenario Background:

You are the lead AI architect for NeuroLink Corp, a company that digitizes human memories into "Engrams." When a user uploads their daily memories to the cloud, the data stream is chaotic. Some memories are corrupted, while others are highly classified.

You need to build a secure pipeline that filters out corrupted or restricted memories, translates the safe memories into a readable digital format, and returns the compiled list of safe memory strings to the user's dashboard.

Implementation Constraints & Requirements:

1. Inheritance and Memory Structure:
   Create an abstract base class MemoryEngram. It must contain:
   String engramId
   double clarityScore
   boolean isCorrupted
   Create two concrete classes that extend MemoryEngram:
   StandardEngram: (Nothing special, inherits fields).
   ClassifiedEngram: Contains an additional field int securityClearanceLevel.
2. Custom Functional Interfaces:
   You must define two custom functional interfaces:
   @FunctionalInterface EngramValidator: Must contain a method that takes a MemoryEngram object and returns a boolean (indicating if the memory is safe to process).
   @FunctionalInterface EngramTranslator: Must contain a method that takes a MemoryEngram object and returns a String (representing the translated text).
3. The Processing Engine (Stream Pipeline):
   Create a class CortexProcessor with a method:
   public List<String> processMemories(List<MemoryEngram> engrams, EngramValidator validator, EngramTranslator translator)
   Inside this method, you must write a single, chained Java Stream pipeline that:
   Filter: Uses the provided EngramValidator to filter out restricted/corrupted memories.
   Filter: Filters out any engram that has a clarityScore less than 50.0 (too blurry to read).
   Map: Uses the provided EngramTranslator to transform the remaining engrams into String format.
   Collect: Returns the final list of translated strings.

4. The Business Logic Rules (Main Execution):
   In your main method, instantiate the custom functional interfaces using Lambda Expressions:
   The Validator Lambda: An engram is unsafe (returns false) if isCorrupted == true OR if it is an instance of ClassifiedEngram where securityClearanceLevel is greater than 3. All other engrams are safe.
   The Translator Lambda: The output string must be formatted exactly as: "ENGRAM-[ID] | CLARITY: [Score]%".

Expected Solution & Detailed Explanation

Why this makes a great interview/study question:

- **Domain-Driven Lambda Injection:** It demonstrates how passing behaviors (Lambdas) into a processing engine allows the core engine (CortexProcessor) to remain completely ignorant of the business rules, ensuring the Open/Closed Principle.
- **Safe Type Casting:** The Validator requires the developer to use instanceof and safe downcasting within a functional block to access child-specific fields.

Coding Problem 3: The "AstroDyne" Thruster Telemetry Analyzer

Scenario Background:

You are programming the diagnostic systems for AstroDyne Corporation's deep-space thrusters. During engine burns, the thruster arrays generate thousands of telemetry logs per minute.

You must construct a pipeline that sifts through these logs, isolates critical heat spikes, formats the data, and returns a purely mathematical stream. To save memory onboard the ship's computer, you must transition from an Object stream to a Primitive stream during the mapping phase.

Implementation Constraints & Requirements:

1. Inheritance and Log Structure:
   Create an abstract base class EngineLog. It must contain:
   String timestamp
   double coreTemperature
   boolean isAnomaly
   Create two concrete classes that extend EngineLog:
   NominalLog: (Inherits fields).
   CriticalLog: Contains an additional field String errorCode.
2. Custom Functional Interfaces:
   Define two custom functional interfaces:
   @FunctionalInterface LogAuditor: Must contain a method taking an EngineLog and returning a boolean.
   @FunctionalInterface HeatExtractor: Must contain a method taking an EngineLog and returning a double.
3. The Processing Engine (Stream Pipeline):
   Create a class TelemetryProcessor with a method:
   public double getPeakCriticalTemp(List<EngineLog> logs, LogAuditor auditor, HeatExtractor extractor)
   Inside this method, write a single chained Stream pipeline that:
   Filter: Uses the LogAuditor to keep only logs deemed critical.
   Map: Uses the HeatExtractor inside the .mapToDouble() method to convert the Stream<EngineLog> into a DoubleStream.
   Terminal Operation: Uses .max() to find the highest temperature, returning 0.0 if no critical logs exist.
4. The Business Logic Rules (Main Execution):
   The Auditor Lambda: A log is critical if isAnomaly == true OR if it is an instance of CriticalLog where the errorCode equals "OVERHEAT".
   The Extractor Lambda: Simply returns the coreTemperature of the log.

Expected Solution & Detailed Explanation

Why this makes a great interview/study question:

- **Primitive Streams:** Converting Stream<Object> to DoubleStream via mapToDouble() is a critical optimization technique. It prevents the JVM from constantly auto-boxing and unboxing Double objects in memory, which is essential for massive telemetry processing.
- **OptionalDouble Handling:** Using .max() on a primitive stream returns an OptionalDouble. The developer must prove they know how to safely unpack it using .orElse(0.0).

Coding Problem 4: The "ChronoCorp" Temporal Paradox Detector

Scenario Background:

You are coding the timeline stabilization software for ChronoCorp, a time-travel agency. Time travelers frequently alter history, creating nested "Temporal Events" containing "Entities".

Your task is to analyze a complex, nested timeline structure. You must flatten out the historical events, filter out entities that do not belong in their current time period, map those entities to a threat report format, and return the final list of paradoxes to the Time Cops.

Implementation Constraints & Requirements:

1. Inheritance and Nested Structure:
   Create an abstract base class TemporalEntity. It must contain:
   String entityName
   int originYear
   Create two concrete classes:
   HumanEntity: Inherits fields.
   ArtifactEntity: Contains an additional boolean isRadioactive.
   Note: Create an HistoricalEvent class that contains a List<TemporalEntity> entities and an int eventYear.
2. Custom Functional Interfaces:
   Define two custom functional interfaces:
   @FunctionalInterface ParadoxChecker: Takes a TemporalEntity and an int (the year of the event), returning a boolean.
   @FunctionalInterface ThreatMapper: Takes a TemporalEntity and returns a String.
3. The Processing Engine (Stream Pipeline):
   Create a class ParadoxAnalyzer with a method:
   public List<String> detectParadoxes(List<HistoricalEvent> timeline, ParadoxChecker checker, ThreatMapper mapper)
   Inside this method:
   FlatMap: You must flatten the List<HistoricalEvent> into a stream of TemporalEntity objects. Note: Since the ParadoxChecker needs the eventYear, you cannot just flatmap the entities blindly. You must process them logically.
   Filter: Use the ParadoxChecker.
   Map: Use the ThreatMapper.
   Collect: Return the List<String>.

4. The Business Logic Rules (Main Execution):
   The Checker Lambda: An entity is a paradox (returns true) if its originYear is greater than the eventYear (it is from the future).
   The Mapper Lambda: Formats as "[Name] detected out of time!".

Expected Solution & Detailed Explanation

Why this makes a great interview/study question:

- **Advanced flatMap usage:** Often, developers only use flatMap to blindly flatten List<List<T>>. This problem forces them to maintain the context of the parent object (eventYear) while mapping the child objects, representing advanced Stream comprehension.

Coding Problem 5: The "GeneWeaver" DNA Sequencer

Scenario Background:

You are processing genetic data for GeneWeaver Labs. The lab sequences thousands of DNA samples. You must build a functional pipeline that filters out unviable samples, maps the viable ones to their genetic classifications, and then groups them together by their species type.

Implementation Constraints & Requirements:

1. Inheritance and Cargo Structure:
   Create an abstract base class DNASample. It must contain:
   String sampleId
   double purityPercentage
   Create two concrete classes:
   HumanSample: Contains String bloodType.
   AlienSample: Contains boolean isSiliconBased.
2. Custom Functional Interfaces:
   Define two custom functional interfaces:
   @FunctionalInterface ViabilityChecker: Takes a DNASample and returns a boolean.
   @FunctionalInterface GenomeMapper: Takes a DNASample and returns a String (representing a classification label).
3. The Processing Engine (Stream Pipeline):
   Create a class Sequencer with a method:
   public Map<String, List<String>> classifyGenomes(List<DNASample> samples, ViabilityChecker checker, GenomeMapper mapper)
   Inside this method, write a single Stream pipeline that:
   Filter: Uses the ViabilityChecker to remove bad samples.
   Collect & Map: Instead of mapping in the middle of the stream, you MUST use Collectors.groupingBy() to group the samples by their Class name (e.g., "HumanSample" or "AlienSample"). Inside the groupingBy, use Collectors.mapping() to apply the GenomeMapper to transform the values in the map into List<String>.
4. The Business Logic Rules (Main Execution):
   The Checker Lambda: A sample is viable if purityPercentage >= 80.0.
   The Mapper Lambda: Returns a string combining the sampleId and its specific child trait (e.g., "ID: 001 (Type: O-)" or "ID: 002 (Silicon: true)").

Why this makes a great interview/study question:

- **Complex Collectors Logic:** Grouping data into a Map is common, but applying a Collectors.mapping() as a downstream collector separates junior developers from senior ones. It allows transforming data *after* the grouping structure has been decided.</String,></List

Problem 1: The E-Commerce Nested Payload Parser (flatMap & Data Aggregation)

Enterprise Scenario: Your Spring Boot microservice receives a massive, deeply nested JSON payload representing multiple Order objects. Each Order contains a list of Item objects. Some items are corrupted (null or negative price).

Task: Write a declarative Stream pipeline that extracts all valid items across all orders, calculates the total revenue generated strictly by items in the "ELECTRONICS" category, and ensures the pipeline throws absolutely no NullPointerExceptions.

Line-by-Line Breakdown:

- .filter(Objects::nonNull): Stops the corrupted root null order from triggering a crash downstream.
- .flatMap(order -> order.getItems().stream()): The critical step. Instead of mapping an Order to a Stream<Item> (which would create a Stream<Stream<Item>>), flatMap extracts all items from all orders and merges them into one continuous, 1-dimensional Stream<Item>.
- .filter(item -> "ELECTRONICS".equals(item.getCategory())): Safe string comparison. Putting the literal "ELECTRONICS" first prevents a NullPointerException if getCategory() returns null.
- .mapToDouble(Item::getPrice): Converts to a DoubleStream, preventing the JVM from wasting memory on wrapper class auto-boxing during the mathematical aggregation.

Problem 2: The Multi-Tiered Audit Log Merger (Custom toMap Resolvers)

Enterprise Scenario: A security microservice parses raw login logs. A single user might log in from multiple IP addresses. You need to map each User ID to a Set of their unique IP addresses.

Task: Transform a List<LoginLog> into an immutable Map<String, Set<String>> where the key is the userId and the value is a unique collection of IP addresses. If a duplicate userId key is encountered during the stream, use a merge function to combine their IP addresses rather than crashing with an IllegalStateException.

Line-by-Line Breakdown:

- .collect(Collectors.toUnmodifiableMap(...)): Creates an enterprise-grade, thread-safe, read-only Map.
- LoginLog::getUserId: The first argument defines the Map's Key.
- log -> new HashSet<>(...): The second argument defines the Map's Value. We wrap the initial IP in a HashSet to guarantee uniqueness.
- (existingSet, newSet) -> { ... }: The third argument is the BinaryOperator merge function. Without this, processing the second "U-101" log would throw a DuplicateKeyException. This logic merges the new IP into the existing Set.

Problem 3: Short-Circuit Optimization Proof (limit & Lazy Evaluation)

Enterprise Scenario: You have a massive dataset of 1 million transaction records. You only need to find the first 3 transactions that exceed $10,000 to trigger a fraud alert. A junior developer writes a stream that filters the entire million-record dataset, sorts it, and *then* takes the first 3.

Task: Write a highly optimized stream pipeline that strictly evaluates only what is necessary using short-circuiting. Use the peek() method to mathematically prove to the console that the JVM stops processing elements the exact millisecond it finds 3 matches.

Line-by-Line Breakdown:

- .peek(...): A non-interfering operation used purely for debugging/auditing. It proves that streams process vertically (one element goes through the whole pipeline) rather than horizontally.
- .filter(amt -> amt > 10000.0): Evaluates the transaction.
- .limit(3): The short-circuiting operation. Once 18000.0 passes the filter (the 3rd match), the JVM permanently terminates the stream. 25000.0 and 400.0 are never even inspected, saving massive CPU cycles on large datasets.

Problem 4: Functional Interface Chaining (Rule Engine Construction)

Enterprise Scenario: A banking API registers new users. The validation logic is complex: users must be over 18, have an active status, and their email must contain "@enterprise.com".

Task: Do not use a stream. Instead, instantiate three separate Predicate<User> objects. Chain them together using .and() to form a single, cohesive CompositeRuleEngine. Apply this composite predicate to evaluate a test user.

Line-by-Line Breakdown:

- Predicate<BankUser> isAdult = ...: By extracting conditions into isolated functional interfaces, we make the business logic highly modular and independently unit-testable.
- isAdult.and(isActive).and(isCorporate): Utilizes default methods provided inside the Predicate interface. It fuses three separate SAM (Single Abstract Method) implementations into one overarching mathematical function, cleanly avoiding messy nested if-else blocks.
- .test(validUser): The single execution call that triggers the entire evaluation chain.

Problem 5: The Thread-Safe State Mutation Trap (reduce Mechanics)

Enterprise Scenario: You are migrating a legacy system that calculates the total inventory count. The junior developer tried to use .parallelStream() combined with an external, mutable Counter object. The result is completely different every time the code is executed due to thread race conditions.

Task: Write the broken parallel stream code that proves the race condition exists. Then, rewrite the exact same logic using the pure, side-effect-free .reduce() terminal operation to guarantee thread-safe parallel aggregation.

Line-by-Line Breakdown:

- .parallelStream().forEach(...): The JVM spawns multiple threads to process the list simultaneously. Because badCounter.add() is not synchronized, multiple threads read and write to total at the exact same microsecond, overwriting each other's work.
- .reduce(0, (acc, curr) -> acc + curr): The declarative solution. 0 is the identity (starting value). The lambda function is pure—it has no side effects and alters no external state. It takes two inputs, adds them, and returns a new immutable value. The JVM safely orchestrates the parallel chunks and combines the final results accurately.